📘 Cahier des Charges

Cours : Programmation Web 2

Niveau : 2ème année E-Business

Technologie principale : PHP Orienté Objet (sans framework)

Environnement : WAMPServer (ou XAMPP ou EasyPHP)

Date limite du dépôt GitHub: Samedi 07/03/2026

1️⃣ Contexte du projet

Dans le cadre du module **Programmation Web 2**, vous devez développer une **application web dynamique complète** en utilisant **PHP orienté objet** et **MySQL**, installés via **WAMPServer**.

⚠️ **L'utilisation de frameworks PHP est strictement interdite** (Laravel, Symfony, CodeIgniter, CakePHP, etc.).
Le projet doit être développé **entièrement en PHP natif orienté objet**.

L'objectif est de vous permettre de :

- Comprendre la programmation orientée objet en PHP
- Structurer un projet web dynamique
- Manipuler une base de données relationnelle
- Implémenter une architecture MVC simple
- Gérer la sécurité d'une application web

---

2 Présentation du projet à réaliser

💡 Nom du projet : ChallengeHub

Plateforme collaborative de défis créatifs

Vous devez développer un site web dynamique permettant à des utilisateurs de :

- Créer un compte
- Publier des défis créatifs
- Participer aux défis publiés par d'autres
- Commenter et voter pour les meilleures participations
- Consulter un classement des meilleures contributions

Il ne s'agit pas d'un simple blog, ni d'un site vitrine. Il s'agit d'une **application web interactive multi-utilisateurs avec gestion de relations complexes en base de données**.

---

3 Fonctionnalités obligatoires

A. Gestion des utilisateurs

- Inscription (nom, email, mot de passe)
- Connexion / Déconnexion
- Gestion de session
- Hashage sécurisé du mot de passe (password_hash)
- Modification du profil
- Suppression de compte

B. Gestion des défis

Chaque utilisateur connecté peut :

- Créer un défi
- Modifier son propre défi
- Supprimer son propre défi
- Définir :
    - Titre
    - Description
    - Catégorie
    - Date limite
    - Image (optionnel)

Les défis doivent être visibles publiquement.

C. Participation aux défis

Un utilisateur peut :

- Soumettre une participation à un défi
- Modifier ou supprimer sa participation
- Ajouter :
    - Description
    - Image ou lien externe

D. Commentaires

- Les utilisateurs connectés peuvent commenter une participation
- Les commentaires affichent :
    - Nom de l’auteur
    - Date
    - Contenu

E. Système de vote

- Un utilisateur peut voter une seule fois par participation
- Les votes doivent être enregistrés en base
- Un classement doit afficher les participations les mieux notées

---

F. Recherche et filtrage

- Recherche par mot-clé
- Filtrage par catégorie
- Tri par popularité ou date

4️⃣ Contraintes techniques obligatoires

🔒 Interdictions

❌ Aucun framework PHP (Laravel, Symfony, CodeIgniter…)
❌ Aucun CMS (WordPress, Joomla…)
❌ Aucun générateur automatique de code
❌ Aucun backend NodeJS ou autre technologie serveur

✅ Obligations techniques

- PHP orienté objet uniquement
- Séparation logique du code (MVC simplifié)
- Connexion PDO obligatoire
- Validation des formulaires
- Protection contre :
  - Injection SQL
  - XSS
  - CSRF (bonus)
- Utilisation de sessions sécurisées

5️⃣ Architecture du projet

Votre projet doit respecter une structure similaire à :

```
/app
  /controllers
  /models
  /views
```

```
/config
/public
  /css
  /js
  /images
index.php
```

Vous devez :

- Créer des classes pour chaque entité principale :
    - User
    - Challenge
    - Submission
    - Comment
    - Vote
- Utiliser des méthodes CRUD propres et organisées
- Centraliser la connexion à la base de données

---

6 Base de données minimale

Tables obligatoires :

- users
- challenges
- submissions
- comments
- votes

Vous devez fournir :

- Script SQL complet

- Diagramme relationnel

---

7 Livrables

Chaque étudiant doit remettre :

1. 📦 Projet complet (dossier compressé)
2. 📄 Rapport technique comprenant :
    - Description du projet
    - Diagramme UML (diagramme de classes minimum)
    - Schéma de base de données
    - Explication de l’architecture MVC
3. 🎥 Démonstration fonctionnelle (présentation le jour du DS1)

---

8 Critères d’évaluation

Critère	Pondération
Architecture POO	25 %
Fonctionnalités complètes	25 %
Qualité du code	15 %
Sécurité	15 %
Interface utilisateur	10 %
Documentation	10 %

9 Fonctionnalités bonus (facultatif)

- Système de badges
- Notifications AJAX
- Pagination dynamique
- API REST interne
- Tableau de bord administrateur

---

10 Objectifs pédagogiques finaux

À la fin du projet, l'étudiant doit être capable de :

- Concevoir une application web dynamique complète
- Appliquer correctement la POO en PHP
- Concevoir et interroger une base de données relationnelle
- Structurer un projet professionnel sans framework
- Comprendre les principes fondamentaux des frameworks (sans les utiliser)